

Verteilte Systeme, Containerisierung

Leistungsbeurteilung VSC-01

Autor / Co-Autor	Patrick Michel
Release	Sommer / Herbst 2026

Notengebung

Leistungsbeurteilung	Prüfungsform	Gewichtung
VSC-01	Transferarbeit / Projekt	60 %
VSC-02	Schriftliche Theorieprüfung	40 %

Personalien

Person 1	
Person 2 (optional)	

Punkte

Maximal erreichbare Punkte	18 Punkte + bis zu 2 Zusatzpunkte
-----------------------------------	--

Prüfungsform	Transferarbeit / Projektarbeit; Einzel- oder Partnerarbeit
Zeitraum und Abgabetermin	Projektstart: 10.08.2026 / Abgabe: 21.09.2026, 23:59 Uhr
Hilfsmittel	Open Book; Kursunterlagen, Dokumentationen und eigene Notizen sind erlaubt
Weitere Vorgaben	Abgabe gemäss Abschnitt "Geforderte Abgaben"; lauffähige Demo ist Teil der Beurteilung

Ehrenkodex: Die eingereichte Lösung muss nachvollziehbar selbst erarbeitet sein. Hilfsmittel, KI-Unterstützung und übernommene Codebestandteile sind transparent im Repository zu dokumentieren.

Ausgangslage und Auftrag

Dispatch City ist ein visuelles Food-Delivery-System, das schrittweise von einer lokalen Container- und Kubernetes-Grundlage zu einem verteilten System mit Messaging, Persistenz, Skalierung und Monitoring ausgebaut wird.

Ihre Aufgabe ist es, die vorbereiteten Services als eigene Container-Images zu bauen, in einem lokalen k3d-Kubernetes-Cluster zu deployen und die Architektur über mehrere Ausbaustufen lauffähig zu machen. Das Dashboard soll in jeder Stufe nutzbar bleiben und den technischen Fortschritt sichtbar machen.

Zielbild: Am Ende läuft Dispatch City als Kubernetes-Anwendung mit Nuxt-Dashboard, Go-Services, RabbitMQ-Event-Pipeline, CloudNativePG-PostgreSQL und Prometheus/Grafana-Observability.

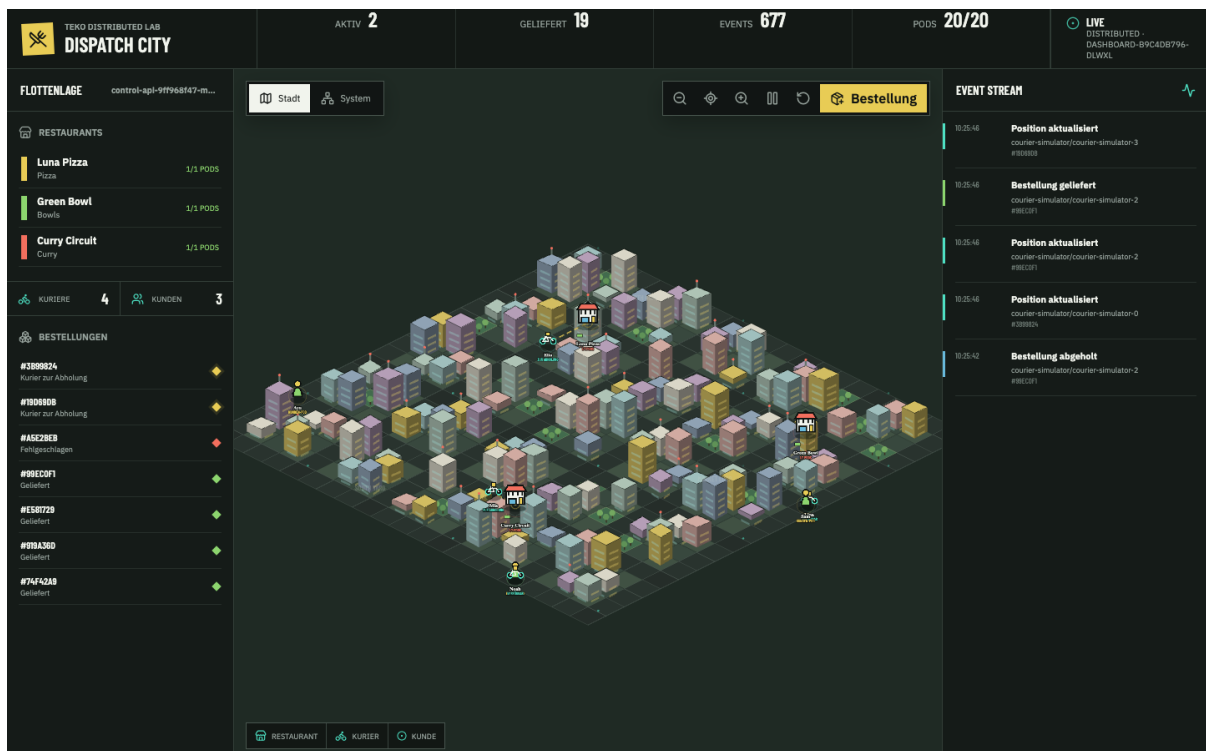


Abbildung 1: Dispatch City als visuelles 2.5D-Dashboard der Referenzimplementation.

Systemidee

- Kundinnen und Kunden erzeugen Bestellungen als Events.
- Restaurant-Worker nehmen Bestellungen an oder lehnen sie ab.
- Courier-Simulatoren bewegen sich sichtbar durch die Stadt und liefern Bestellungen aus.
- Der Order Worker verarbeitet Events, hält den fachlichen Zustand konsistent und persistiert ihn später in PostgreSQL.
- Das Dashboard zeigt fachliche Vorgänge und technische Kubernetes-Zustände in einer gemeinsamen Live-Ansicht.

Zielarchitektur und Komponenten

Dispatch City Zielarchitektur

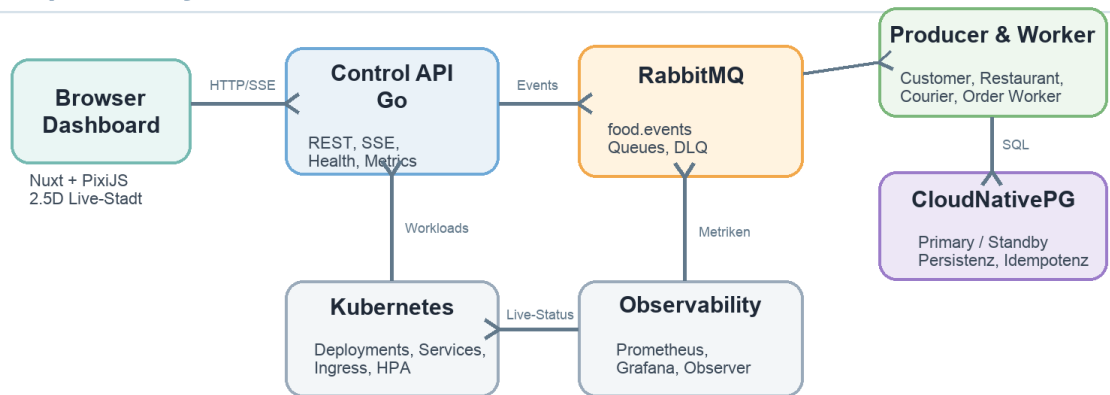


Abbildung 2: Zielarchitektur der kumulativen Projektarbeit.

Komponente	Technologie	Aufgabe im Projekt
dashboard	Nuxt, PixiJS, eigenes Image	2.5D-Stadt, Systemansicht, Steuerung und Live-Events via REST/SSE.
control-api	Go, eigenes Image	API für Snapshot, Steuerung, Health, Metrics und anfänglich In-Memory-Simulation.
RabbitMQ	Standard-Image / Helm	Topic Exchange, Queues, Retry/DLQ und Entkopplung von Producer/Consumer.
customer-simulator	Go, eigenes Image	Erzeugt Kundenidentitäten und Bestellungen als Events.
restaurant-worker	Go, eigenes Image	Verarbeitet Bestellungen als konkurrierende Kitchen-Consumer.
courier-simulator	Go, eigenes Image	Simuliert stabile Kurier-Pods, Fahrten und Zustellungen.
order-worker	Go, eigenes Image	Konsolidiert Events, schreibt Projektion und sorgt für Idempotenz.
PostgreSQL	CloudNativePG	Primary/Standby-Cluster für persistente verteilte Datenhaltung.
Monitoring	kube-prometheus-stack	Prometheus, Grafana, Metriken, HPA und technische Transparenz.

Ausbaustufen

Block	Fokus	Umsetzung	Sichtbares Ergebnis
3	Kubernetes Foundation	Dashboard und Control API als eigene Images, Deployments, Services, ConfigMaps, Probes.	Port-Forward auf Dashboard/API; animierte Stadt läuft standalone.
4	Ingress und Load Balancing	Traefik-Ingress, gemeinsame URL, zwei Dashboard-Replikas, API weiterhin singleton.	Dashboard unter http://localhost:8080; Load Balancing und Pod-Ausfall sichtbar.
5	RabbitMQ Event Pipeline	RabbitMQ, Exchanges/Queues, Customer-, Restaurant-, Courier- und Order-Komponenten.	Bestellungen laufen als Events durch Producer und Consumer; RabbitMQ UI zeigt Backlog.
6	CloudNativePG Persistenz	Operator/Cluster, Migration Job, persistente Order-Projektion, idempotente Event-Verarbeitung.	Zustand über Neustarts hinweg stabil; Primary/Standby und Failover nachvollziehbar.
7	Observability & Resilienz	cluster-observer, kube-prometheus-stack, ServiceMonitors, Grafana, HPA, Failure-/Smoke-Skripte.	Fachlicher Flow, Clusterzustand, Metriken und Skalierung sind gemeinsam demonstrierbar.

Daten- und Schnittstellenstruktur

API

- REST-Endpunkte für Snapshot, Steuerung der Simulation und manuelle Bestellungen.
- SSE-Stream für Live-Updates an das Dashboard.
- Health-, Readiness- und Metrics-Endpunkte für Kubernetes und Prometheus.

Event Envelope

Jedes Event enthält mindestens: event_id, event_version, timestamp, correlation_id, causation_id, source und einen typisierten Payload.

Zentrale Events: customer.registered, courier.registered, order.created, order.accepted, order.rejected, courier.assigned, courier.location.updated, order.picked_up, order.delivered, simulation.started, simulation.paused und simulation.reset.

Persistenz

Der Order Worker ist der einzige Schreiber des fachlichen Order-Zustands. PostgreSQL speichert unter anderem Orders, Order Events und processed_events zur idempotenten Verarbeitung von RabbitMQ-Nachrichten.

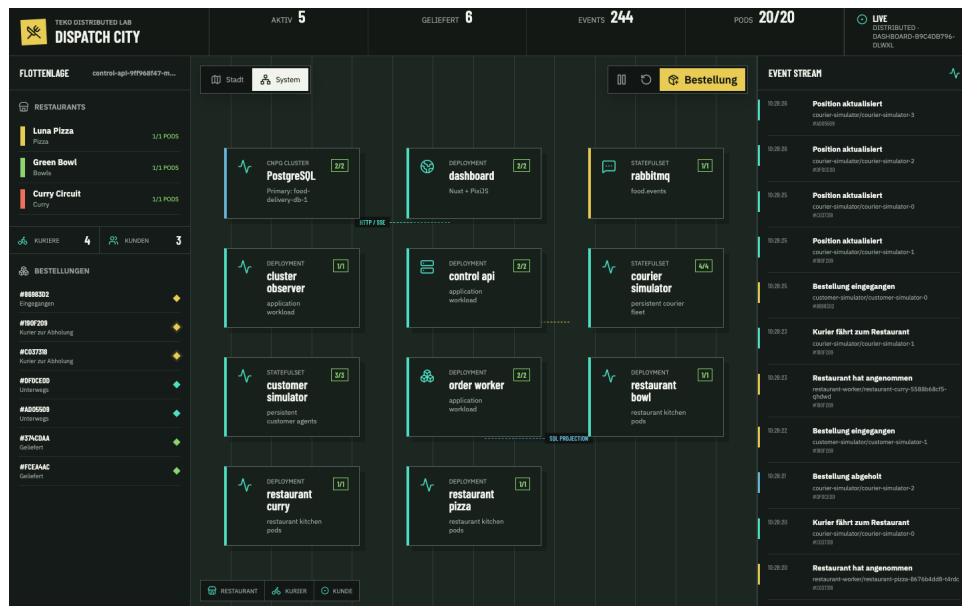


Abbildung 3: Systemansicht der Referenzimplementierung mit Workloads und Readiness.

Rahmenbedingungen

- Der Cluster läuft lokal mit k3d. Die Lösung muss auf macOS und Windows mit Docker Desktop realistisch nachvollziehbar sein.
- Eigene Anwendungen werden aus dem bereitgestellten Code als Docker Images gebaut und in den k3d-Cluster geladen oder über eine Registry verfügbar gemacht.
- Kubernetes-Manifeste sollen strukturiert abgelegt werden, beispielsweise mit Kustomize-Overlays pro Ausbaustufe.
- Standardsoftware wie RabbitMQ, CloudNativePG und kube-prometheus-stack darf als offizielles Image, Operator oder Helm Chart eingesetzt werden.
- Die finale Lösung muss mit dokumentierten Befehlen reproduzierbar deploybar sein.
- Die Arbeit kann als Einzel- oder Partnerarbeit durchgeführt werden. Bei Partnerarbeit müssen Aufgabenverteilung und Beiträge beider Personen im Repository nachvollziehbar sein.
- Zur Live-Abnahme wird eine kurze, nicht vorab bekannte Änderung oder Fehlersituation aus den behandelten Themen bearbeitet. Eingesetzte KI-Hilfsmittel sind im README zu dokumentieren.

Geforderte Abgaben

- Link auf das Git-Repository mit vollständiger Projektlösung und nachvollziehbaren Zwischenständen pro Unterrichtsblock.
- README mit Voraussetzungen, Build-, Deploy-, Reset- und Smoke-Test-Befehlen.
- Dockerfiles, Build-Skripte und Kubernetes-/Kustomize-Manifeste.
- Kurze Architekturdokumentation mit Komponenten, Eventfluss, Datenhaltung und wichtigsten Entscheidungen.
- Nachweis der finalen Demo: Screenshots oder kurzes Video von Dashboard, RabbitMQ, PostgreSQL/CloudNativePG und Grafana.
- Kurze Reflexion: bekannte Grenzen, beobachtete Fehlerbilder und mögliche Verbesserungen.

Abgabe: Die Abgabe erfolgt bis 21.09.2026, 23:59 Uhr. In der Unterrichtseinheit vom 21.09.2026 werden Theorieprüfung und Projektstand besprochen.

Bewertungsraster

Die regulären Anforderungen ergeben maximal 18 Punkte. Zusätzlich können bis zu 2 individuell beurteilte Zusatzpunkte erreicht werden.

Bereich	Kriterien	Punkte
Kubernetes-Grundsystem und Images	Dashboard und Control API als eigene Images gebaut; Deployments, Services, ConfigMaps und Probes funktionieren; Struktur der Manifeste ist nachvollziehbar.	3
Ingress und Load Balancing	Gemeinsamer Einstiegspunkt für Dashboard und API; Dashboard-Replikas und Service-Load-Balancing sind demonstrierbar; API bleibt wegen In-Memory-State korrekt singleton.	2
RabbitMQ Event Pipeline	RabbitMQ mit Exchange/Queues/DLQ; Producer und Consumer laufen; Eventfluss von order.created bis order.delivered ist sichtbar; Acknowledgements, Retry/Backlog und Skalierung werden sauber erklärt.	4
CloudNativePG und Persistenz	PostgreSQL läuft als Primary/Standby-Cluster; Migration und Secrets sind sauber umgesetzt; Order Worker persistiert Projektion und verarbeitet Events idempotent.	3
Observability und Resilienz	Prometheus/Grafana sind per Helm installiert; relevante Metriken, Probes, Ressourcen und HPA sind vorhanden; Failure- und Skalierungsdemo sind nachvollziehbar.	3
Dokumentation, Git und Demo	Repository ist sauber versioniert und enthält nachvollziehbare Zwischenstände; README und Architektur-/Betriebsnotizen ermöglichen Reproduktion; finale Demo, kurze Live-Anpassung und Reflexion sind vollständig. Bei Partnerarbeit sind die Beiträge beider Personen erkennbar.	3
Besondere Zusatzleistung (optional)	Technisch sinnvolle und nachvollziehbar integrierte Erweiterungen, die über die vorgegebenen Anforderungen hinausgehen. Art und Umfang werden durch die Lehrperson individuell beurteilt.	bis 2
Maximal inkl. Zusatzpunkte		20